CS-461, Spring 2026
Homework #2: Question Answering with Transformers
Due Date: Thursday, May 28th @ 11:59PM
Total Points: 15.0 + 1.0 Bonus

Description: Question-answering with transformer-based architectures is the dominant paradigm in natural language processing. Among the various formulations of this task, multiple-choice question-answering is among the most tractable. In this assignment, you will perform question-answering on the OpenBookQA dataset (see https://arxiv.org/abs/1809.02789). OpenBookQA is a multiple-choice question-answering dataset of elementary school-level scientific questions. Questions are posed in natural language with four possible choices. Background knowledge in the form of a one-sentence "fact" is also provided.

This task's primary formulation requires you to identify the correct "fact" from among 1,326 facts. To focus on transformer architectures in this assignment, you are given the "fact" associated with each question. The training dataset consists of approximately 5,000 observations; the validation and test sets each contain 500 observations. Each observation includes a question stem, the associated fact, four labeled choices and a correct answer label. You may ignore all other information.

You will use transformer-based architectures for multiple-choice question-answering, including classification and generative approaches. Starter code is provided to read the dataset files. You may need to install Huggingface Transformers (see https://huggingface.co/docs/transformers/installation) on your machine.

You may discuss the homework with others but do not take any written record from the discussions. You must submit your own work. Do not submit another's work as your own, and do not allow another student to submit your work as their own. Also, do not copy any source code from the Web or use any AI-generated content.

Tasks:

1. **Classification Encoder-OnlyApproach (5.0 pts):** For this approach, you must use the bert-base-uncased checkpoint for the BertModel (see https://huggingface.co/google-bert/bert-base-uncased) and fine-tune the model on the training set using your own code. Implementations using BertForMultipleChoice or other specialized variants will not be accepted. The format of the text input to BERT and the classification methodology for this task are up to you. One approach is to leverage the [CLS] token, which can be implemented as:

    a. Encode each instance as
    ```
    [CLS] <fact> <stem> <text of choice #1> [END]
    [CLS] <fact> <stem> <text of choice #2> [END]
    [CLS] <fact> <stem> <text of choice #3> [END]
    [CLS] <fact> <stem> <text of choice #4> [END]
    ```
    b. Retrieve the context embedding at the top of the transformer stack for the `[CLS]` token and take the dot product with a $[d_{mod} \times 1]$ linear layer.
    c. Treat the resulting vector as logits, apply the softmax and compute a multiclass cross-entropy loss over the four choices.
    d. Fine-tune BERT on the complete dataset with this encoding for multiple epochs.
    e. Use the trained model to perform inference on the validation and test sets.

    Please describe your approach in sufficient detail so that someone familiar with transformers can replicate your results. Also, report your zero-shot and fine-tuned accuracies on the validation and test sets. Please discuss any limitations of your approach and possible solutions. Using the approach described, I achieved an accuracy of 55.9% on the test set.

2. **Classification Decoder-Only Approach (5.0 pts):** For this approach, you will use a GPT-style language model to perform classification. A simple decoder-only transformer-based language model (Aster) pre-trained on a 3GB corpus of encyclopedia, dialog and scientific texts is provided as starter code. Before using Aster, you should validate the model by running the following script to generate the perplexity on the validation set:

```
python Aster.py -loadname saved/model_best.pt -d_model 1024 -d_ff 4096
-n_layers 16 -heads 16 -seqlen 1024 -eval_batchsize 8
-valid_file saved/mixture_valid.txt -tokenizer saved/tokenizer
```

You will fine-tune Aster on the OpenBookQA training set using your own code to perform multi-class classification. The format of the text input to Aster and the classification methodology for this task are up to you. A couple of approaches were discussed in class including using Aster to estimate the probability over text composed as follows:

P("<fact> <stem> <text of choice #1>")
P("<fact> <stem> <text of choice #2>")
P("<fact> <stem> <text of choice #3>")
P("<fact> <stem> <text of choice #4>")

and selecting the highest probability text as the answer. Another approach is to train Aster to emit a single token from {A, B, C, D}. Many other approaches are possible.

Notes: You will need to tokenize your text input to Aster using the tokenize provided with the starter code. Also the `test_model()` function illustrates how to use Aster, a suggested method of feeding data to the model without using a `DataLoader`, and how to calculate the model loss (and associated probabilities without using `F.Corss_Entropy`). These may be beneficial for other parts of the assignment.

Please describe your approach in sufficient detail so that someone familiar with transformers can replicate your results. Also, report your zero-shot and fine-tuned accuracies on the validation and test sets. Please discuss any limitations of your approach and possible solutions. Using the approach described, I achieved an accuracy of 27.6% and 57.6% on the validation set for the zero-shot and fine-tuned models, respectively.

3. **Generative Vanilla Beam Search Approach (5.0 pts):** For this approach, you will implement beam search using the probability distributions generated auto-regressively by Aster. The beam width, sampling methodology, maximum beam length, temperature and other hyper-parameters are up to you. You will fine-tune Aster on the OpenBookQA training set using your own code to generate a multi-token natural language answer. The format of the text input to Aster for this task are up to you. You will also implement your own version of BERTScore on the word embeddings from Aster, and use it at inference to map the generated answer to one of the four multiple choices.

Please describe your approach in sufficient detail so that someone familiar with transformers can replicate your results. Also, report your zero-shot and fine-tuned accuracies on the validation and test sets. Please discuss any limitations of your approach and possible solutions. Using the approach described, I achieved an accuracy of 26.0% and 43.8% on the validation set for the zero-shot and fine-tuned models, respectively.

In addition, please provide five examples of where your model answered correctly and five examples where you model answered incorrectly. Include the `<fact> <stem> <choices>`, `<generation>`, BERTScores, gold answer and predicted answer for each. Comment on the quality of the generations for each group and speculate on the cause of any patterns that you may observe.

4. **Generative Guided Beam Search Approach (1.0 bonus pts):** For the approach you will use *guided beam search* by modifying the vanilla beam search used above to use a *composite score* to make pruning decisions (see NEUROLOGIC https://arxiv.org/abs/2010.12884). Unlike NEUROLOGIC which uses lexical features, you will construct a composite score using a combination of likelihood and the BERTScore between `<fact> + <stem>` and the `<generation>`. Hyper-parameter selection is up to you.

   Note: Since you are modifying only the beam search algorithm, you can use the fine-tuned model from the prior approach.

   Please describe your approach in sufficient detail so that someone familiar with transformers can replicate your results. Also, report your zero-shot and fine-tuned accuracies on the validation and test sets. Please discuss any limitations of your approach and possible solutions. Using the approach described, I achieved an accuracy of 47.6% on the validation set.

   In addition, please provide five examples of where your model answered correctly and five examples where you model answered incorrectly. Include the `<fact> <stem> <choices>`, `<generation>`, BERTScores, gold answer and predicted answer for each. Comment on the quality of the generations for each group and speculate on the cause of any patterns that you may observe.

What to Submit: Please submit a single .zip file for the group. The zip file should contain the following:

- A single .pdf for all written responses and results.
- Individual .py files (two files maximum) for your BERT and Aster approaches, including comments on your implementations.